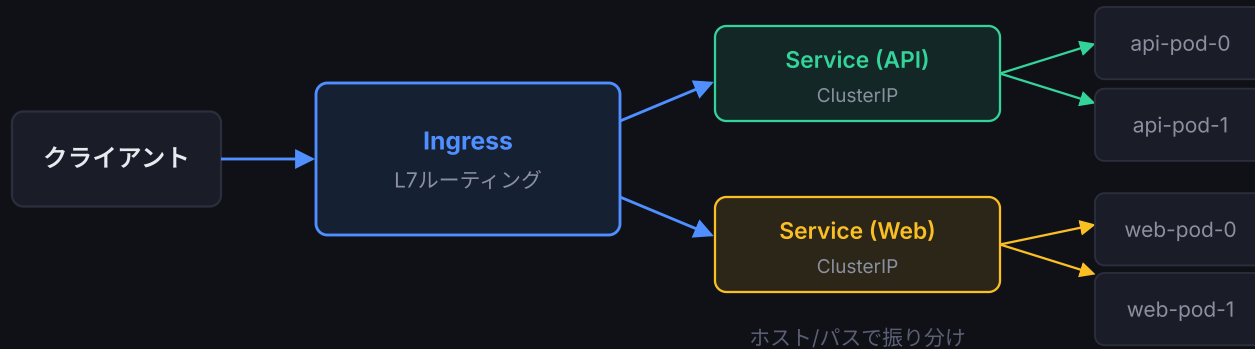


Kubernetes Foundation

KubernetesアプリをService・ Ingressで外部公開する

ClusterIP / NodePort / LoadBalancer・Ingress Controller

ServiceとIngressの2段構えで、PodをL4負荷分散→L7ルーティングで安全に外部公開できる



01

ServiceがPodの窓口を安定させる

PodのIPが変わってもServiceのClusterIPは固定。セクターで自動的に最新Podへ負荷分散される

02

外部公開はIngress+ClusterIPが正解

NodePort/LoadBalancerは学習・暫定用。本番はIngress Controller+ClusterIPでL7ルーティングを実現する

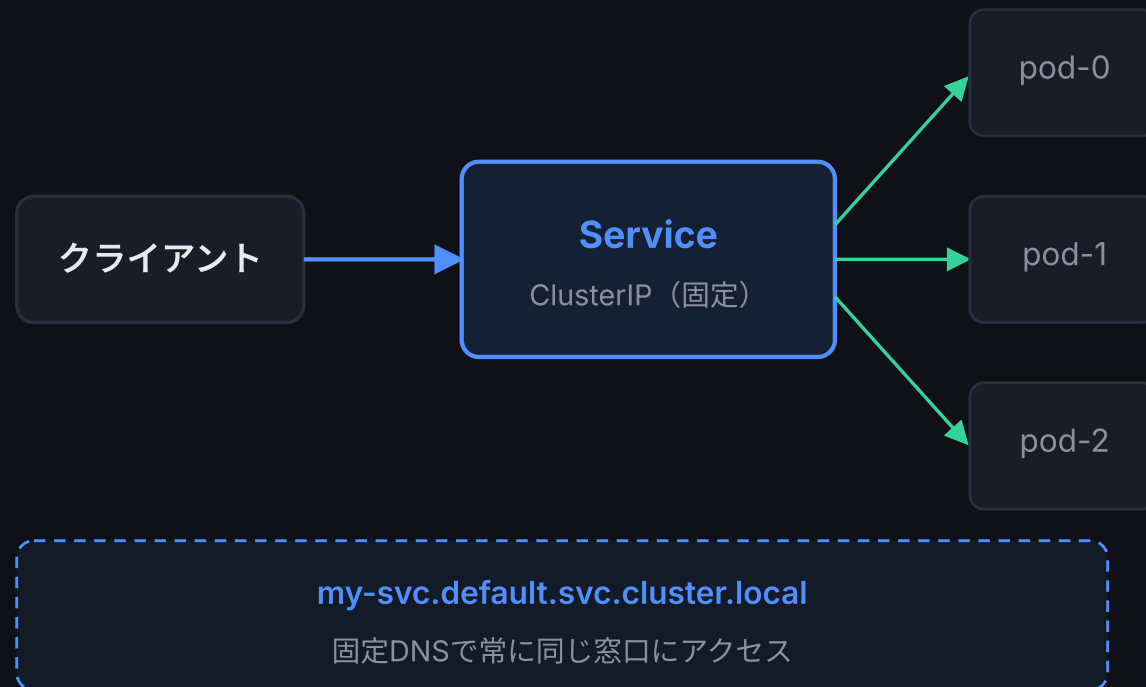
03

Ingressはホスト/パスで振り分け

1つのIngress Controller (Nginx Pod) が複数Serviceヘルルベースで振り分ける。SSL終端も担う

PodのIPが変わる問題をServiceが解決する

- Podは使い捨て前提。再起動のたびにIPが変わるためクライアントは接続先を追えない
- Service (ClusterIP) を置くと、固定の仮想IPと名前でPod群にアクセスできる
- 複数Podが存在する場合はラウンドロビンで自動負荷分散。Podを入れ替えても接続先は変わらない



Serviceには4つのタイプがある

ClusterIP

クラスタ内通信・負荷分散

クラスタ内

Headless

Pod個別DNS / StatefulSet用

NodePort

ノードIP:ポートで外部公開

外部公開

LoadBalancer

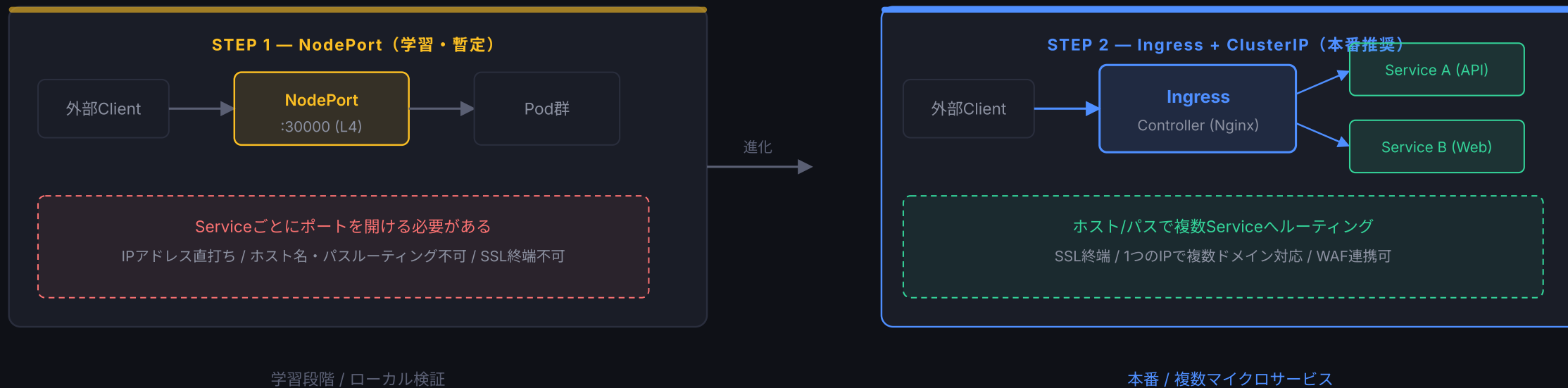
クラウドLBを自動作成

本番推奨: ClusterIP + Ingress Controller

L7ルーティング・SSL終端・複数Service対応

- ClusterIP：クラスタ内専用の仮想IP。**マイクロサービス間通信の標準**
- Headless（ClusterIP: None）：仮想IPなし。**PodのIPをDNSで直接解決**。StatefulSetのDB用
- NodePort：ノードのポート（30000～32767）を開けて外部公開。**学習・暫定用**
- LoadBalancer：クラウドのLBを自動作成して外部公開。コストが高く管理が煩雑

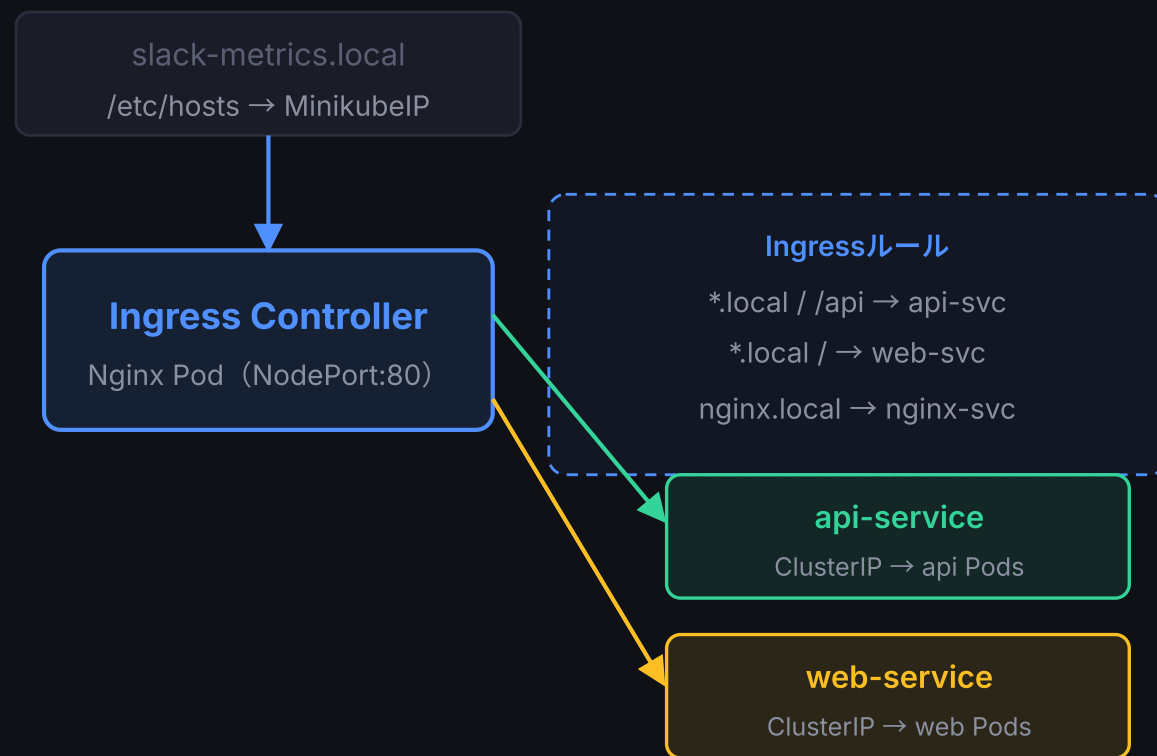
NodePortからIngressへ — 外部公開の進化



IngressがL7でマイクロサービスを振り分ける

- Ingress Controllerの実体は**Nginx**のPod。NodePortで外部に口を開け、内部ではClusterIPへ転送する
- Ingressリソースにホスト・パスのルールを定義。`/api → **api-service**`のようにL7で振り分ける
- **SSL終端**もIngress Controllerが担う。各Podに証明書を配る必要がない

ローカル開発では/etc/hostsでDNSを手動設定。本番ではRoute 53などのDNSと組み合わせる



Service Type — 使い分け基準

タイプ	公開範囲	特徴	使用場面
ClusterIP	クラスタ内のみ	仮想IP固定 / ラウンドロビン負荷分散	マイクロサービス間通信 / DBアクセス
Headless	クラスタ内のみ	仮想IPなし / PodのIPをDNSで返す	StatefulSetのDB / Pod個別アクセス
NodePort	ノードIP経由で外部	ポート30000～32767 / L4(TCP)	ローカル検証 / 学習用
LoadBalancer	クラウドLB経由で外部	クラウドLBを自動作成 / コスト高	小規模・単一Service公開
Ingress	L7で外部（推奨）	ホスト/パスルーティング / SSL終端	本番 / 複数マイクロサービス

まとめ

01

ClusterIPがすべての基本

どのServiceタイプも内部ではClusterIPを使う。Pod群への固定窓口を提供し、ラウンドロビンで負荷分散する

02

外部公開は Ingress + ClusterIP

NodePortは学習用・暫定。本番はIngress Controller (Nginx Pod) にL7ルーティングを集約する

03

IngressはL7の中央集権ルーター

—
ホスト/パスで複数Serviceへ振り分け、SSL終端も担う。1つのControllerで複数マイクロサービスを管理できる